

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет ФТИИ

Дисциплина:
«Frontend-разработка»

Лабораторная № 2

Выполнила:

Кузнецова Таисия
студент группы J3214

(подпись)

Проверила:

...

(отметка о выполнении)

Санкт-Петербург

2026 г.

Цель работы:

Целью работы была разработка и доработка клиентского веб-приложения для планирования путешествий. За основу взят вариант «Приложение для планирования путешествий и маршрутов». На этом этапе проект был связан с внешним mock API, дополнен интерактивными функциями, улучшенной доступностью, двумя темами оформления и общим SVG-спрайтом.

В приложении должны быть реализованы обязательные страницы входа, регистрации, личного кабинета и поиска с фильтрацией. Дополнительно в Travelly предусмотрены страницы направлений, историй путешественников, планировщика маршрута и сохранённых поездок.

Выбранный вариант и концепция Travelly

Travelly это платформа, на которой пользователь может искать направления, сохранять понравившиеся места, составлять маршрут по дням, добавлять заметки на карту и знакомиться с историями других путешественников. Концепция построена вокруг последовательного сценария: поиск идеи, выбор направления планирование поездки сохранение маршрута, обмен опытом.

Используемые технологии

Технология

Назначение

HTML5

Структура страниц, формы, семантические элементы и атрибуты доступности.

CSS3

Стили интерфейса, адаптивность, карточки, состояния элементов и анимации.

Bootstrap 5.3.3

Готовые компоненты и адаптивная сетка.

JavaScript

Обработка форм, фильтрация, лайки, модальные окна, темизация и логика планировщика.

Fetch API

HTTP-запросы к mock REST API.

JSON Server

Имитация backend API на основе файла db.json.

MapTiler SDK

Интерактивная карта и пользовательские маркеры.

localStorage

Сессия пользователя и резервное локальное состояние.

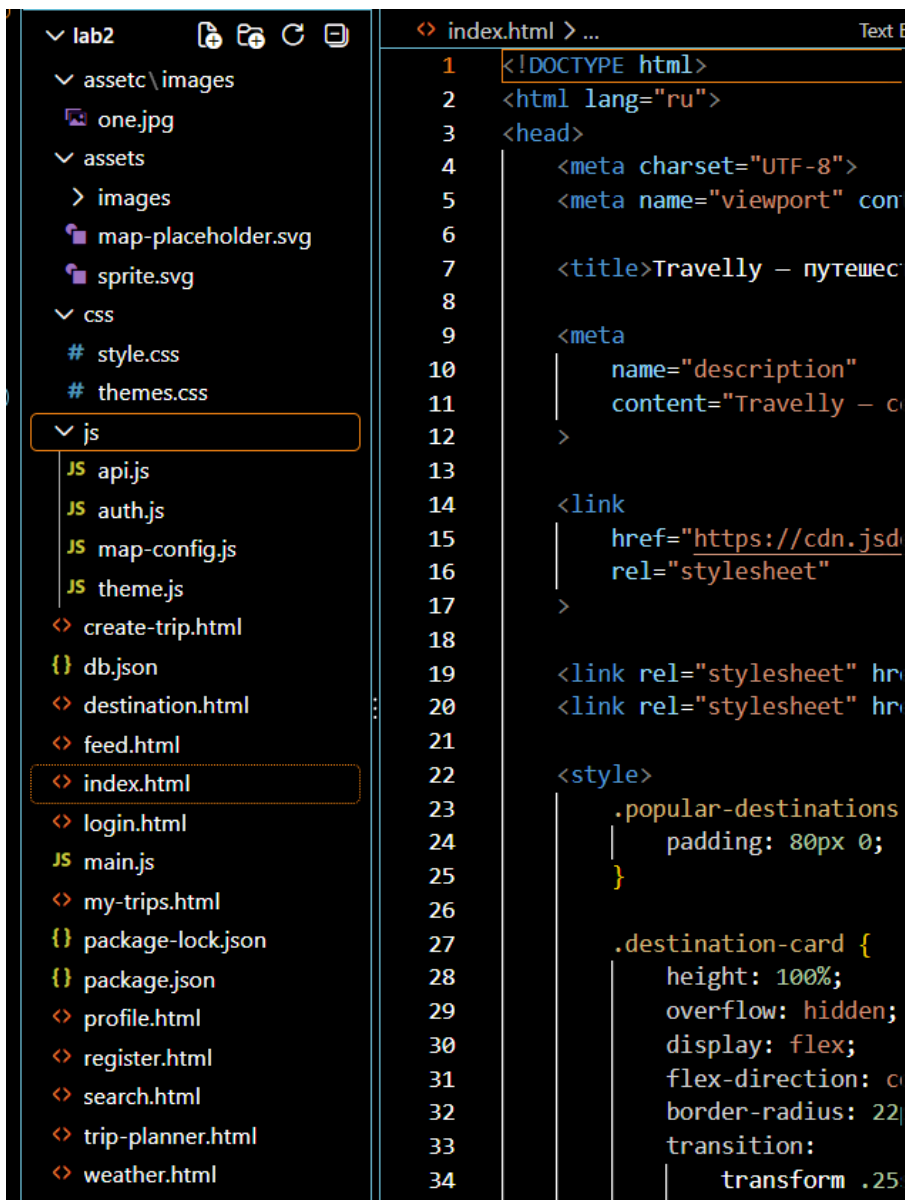
SVG

Общий спрайт интерфейсных иконок.

Структура проекта и страницы

Проект разделён по назначению: HTML-страницы отвечают за представление, CSS-файлы за стили и темы, JavaScript-файлы за общую логику и работу с API, db.json за данные mock-сервера.

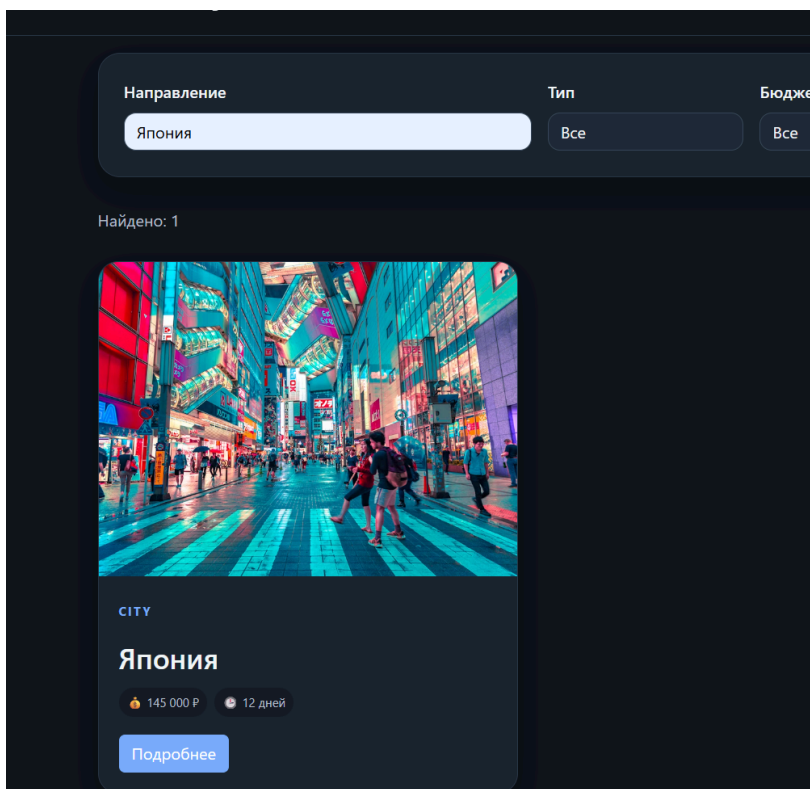
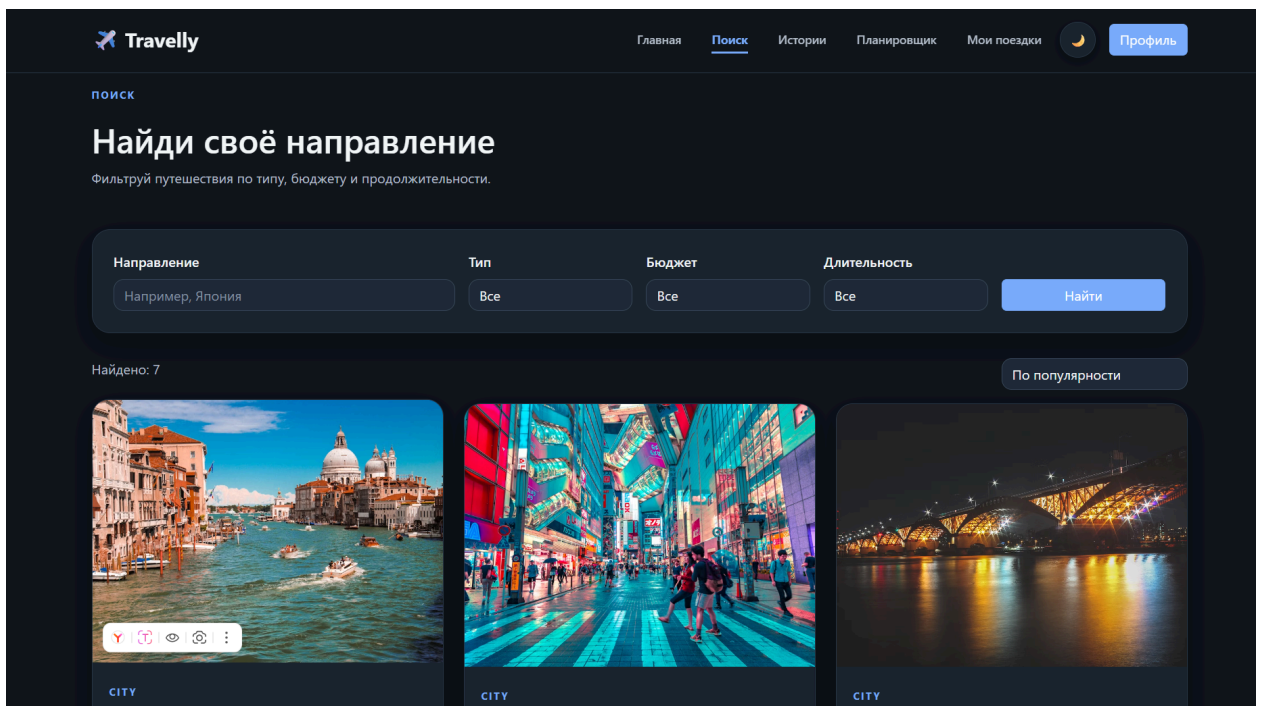
- index.html - главная страница Travelly;
- login.html и register.html - вход и регистрация пользователя;
- profile.html - личный кабинет;
- search.html - поиск и фильтрация направлений;
- destination.html - страница отдельного направления;
- feed.html - истории путешественников и лайки;
- trip-planner.html - планировщик маршрута и карта;
- my-trips.html - сохранённые поездки;
- create-trip.html - создание пользовательской поездки;
- css/style.css и css/themes.css - основные стили и темы;
- js/api.js, js/auth.js, js/theme.js - API, авторизация и темизация;
- js/map-config.js - конфигурация MapTiler;
- assets/sprite.svg - общий SVG-спрайт;
- db.json - данные mock API.



Реализация основных функций

1. Поиск и фильтрация направлений

На странице поиска пользователь может искать направления по названию и использовать фильтры по типу отдыха, бюджету и продолжительности. Данные направлений берутся из коллекции `destinations` через API. Фильтрация выполняется на стороне клиента после получения данных.



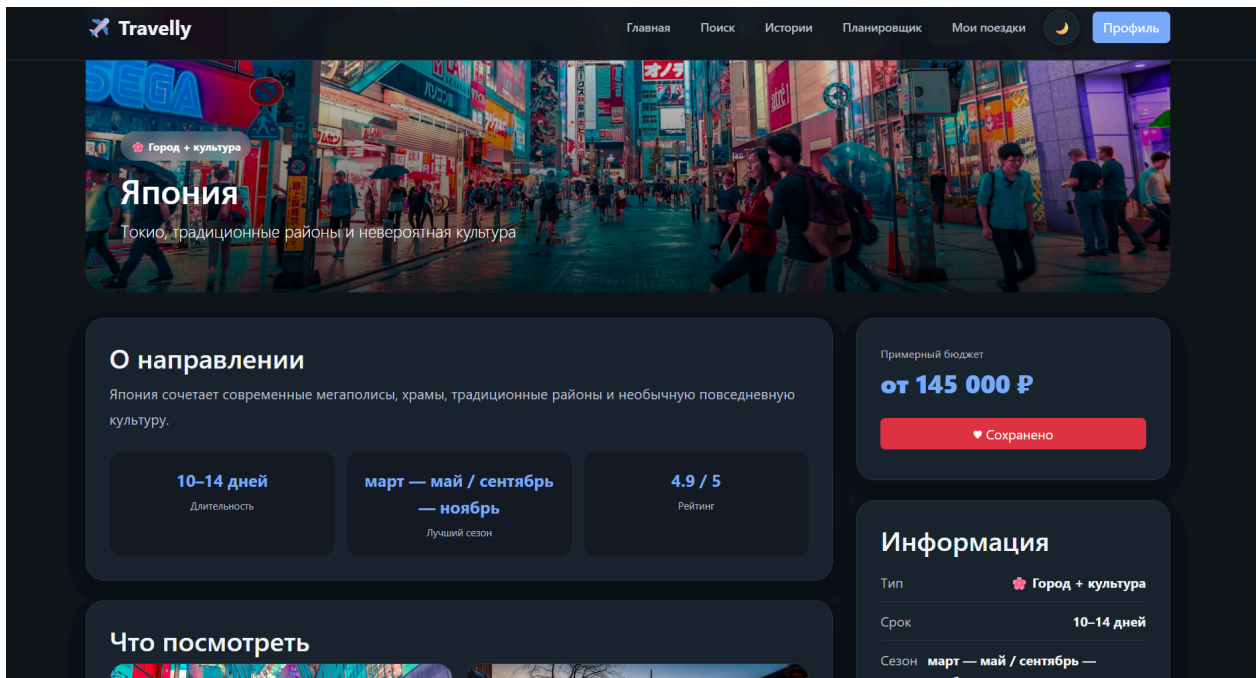
5.2. Страница направления

Для вывода данных используется JavaScript, который получает объект направления и динамически заполняет соответствующие элементы DOM. Благодаря такому подходу одна и та же структура страницы может использоваться для разных направлений без создания отдельных

Санкт-Петербург

2026 г.

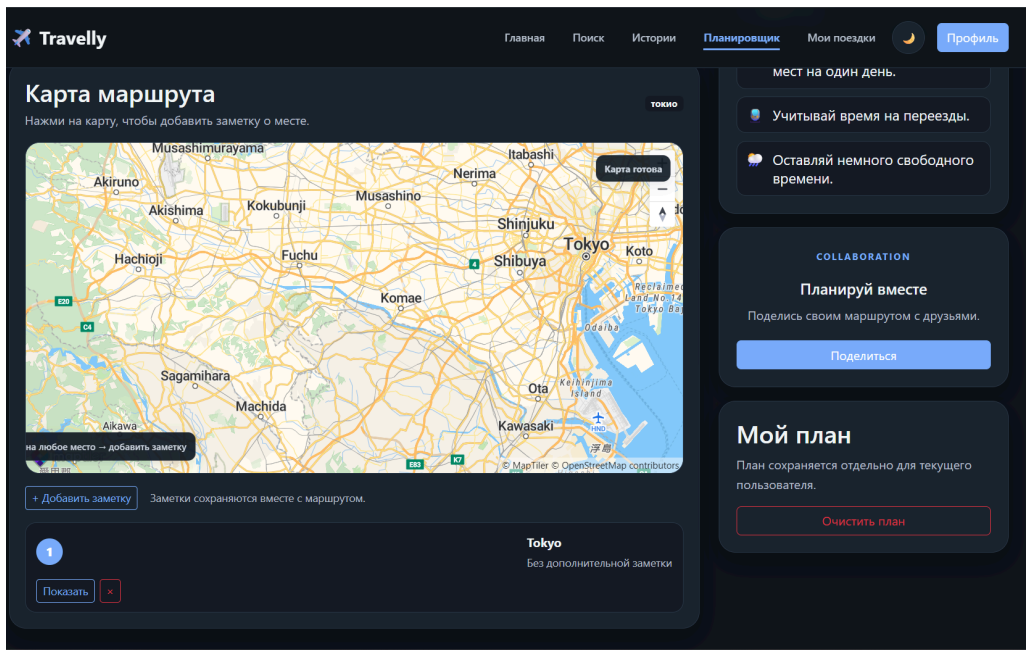
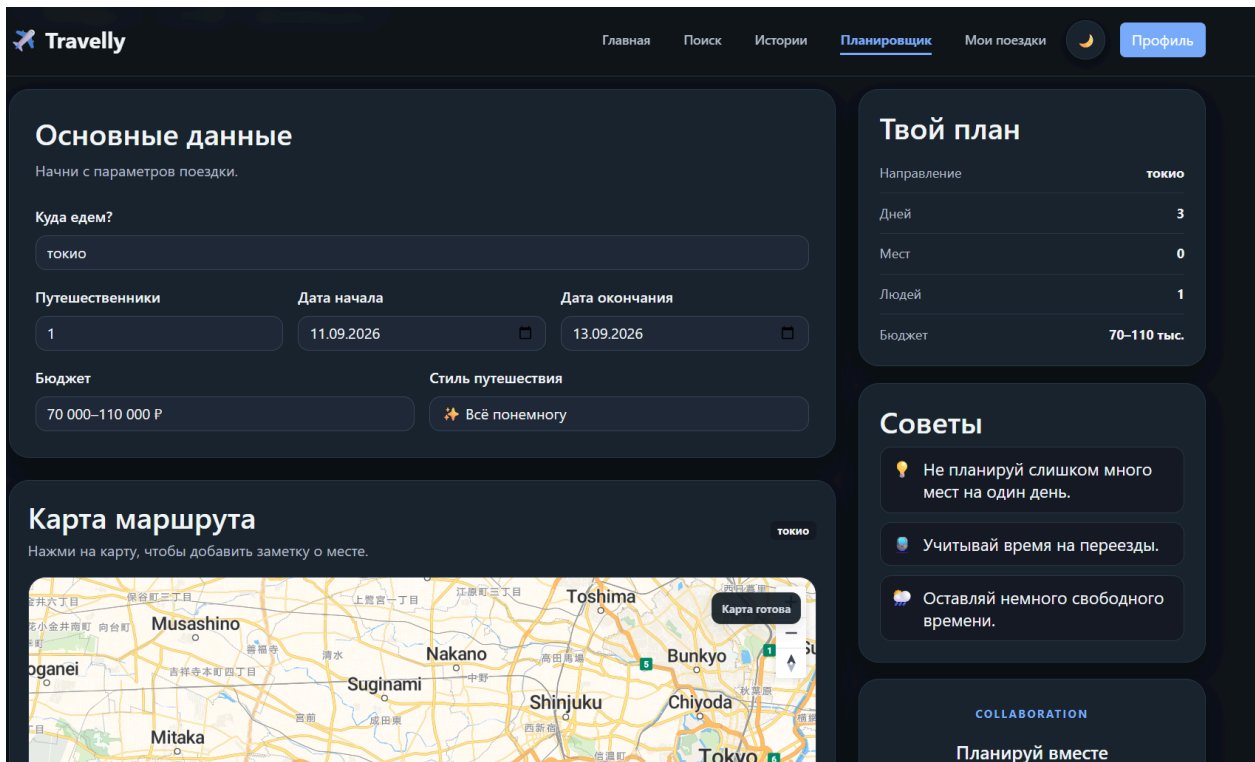
HTML-страниц для каждого города или страны. также реализована функция сохранения направления. При нажатии на кнопку проверяется состояние авторизации пользователя, после чего выбранное направление добавляется в список сохранённых. Для хранения пользовательских данных используется localStorage, при подключённом JSON Server данные также могут передаваться через API. При повторном открытии страницы состояние сохранения восстанавливается, поэтому пользователю не требуется добавлять направление повторно.



5.3. Планировщик маршрута

На странице расположена форма, с помощью которой пользователь задаёт параметры поездки: направление, даты начала и окончания, количество путешественников, бюджет и стиль путешествия. Обработка формы выполняется с помощью JavaScript без перезагрузки страницы. Полученные значения преобразуются в объект маршрута и используются для формирования структуры поездки. Маршрут представлен последовательностью дней, внутри которых пользователь может добавлять отдельные остановки и места. Для динамического добавления элементов используются обработчики событий и создание DOM-элементов. При добавлении нового дня или остановки интерфейс обновляется без перехода на другую страницу. Состояние планировщика сохраняется в localStorage с привязкой к идентификатору авторизованного пользователя. Благодаря этому данные разных пользователей не смешиваются: каждый пользователь

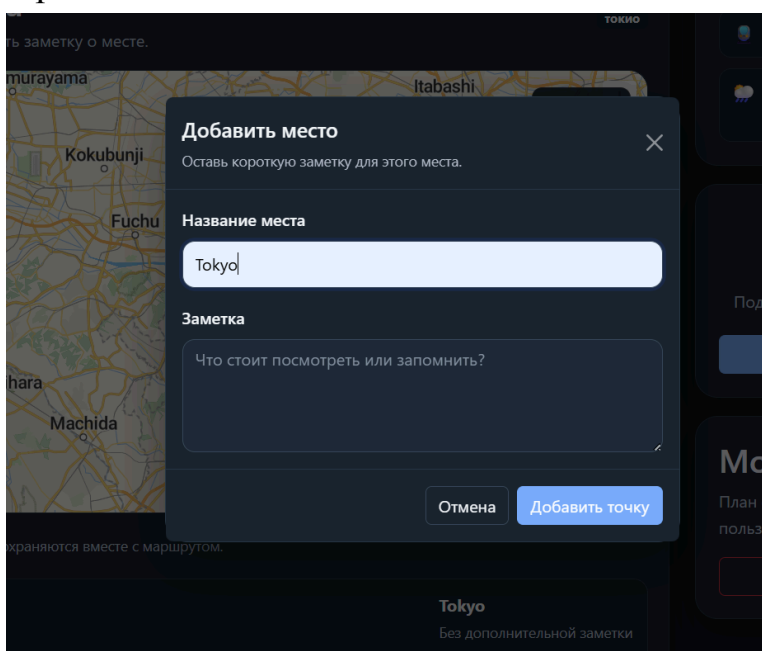
получает собственный набор маршрутов. Для взаимодействия с mock API предусмотрен отдельный модуль `api.js`, через который выполняются HTTP-запросы к JSON Server.



5.4. Заметки на карте

Для работы с заметками реализована интерактивная карта, которая подключается к странице планировщика. Карта используется не только как

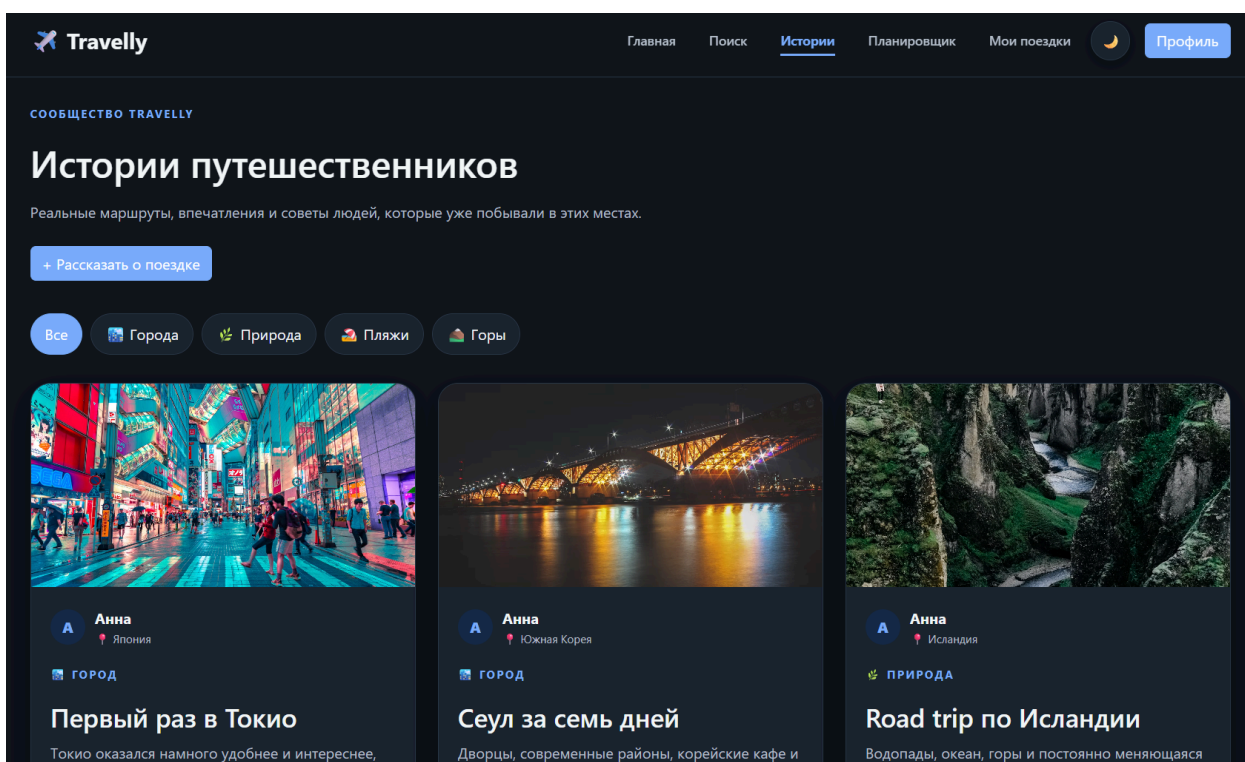
визуальный элемент, но и как средство взаимодействия с данными маршрута. При нажатии пользователя на определённую область карты JavaScript получает координаты события и открывает модальное окно добавления заметки. В модальном окне находятся поля для названия места и дополнительного описания. После отправки формы создаётся объект заметки, содержащий, в частности, название, текст и координаты точки. Объект добавляется в массив заметок текущего маршрута, после чего интерфейс обновляется: на карте появляется маркер, а сама заметка отображается в списке под картой. Для каждой записи предусмотрены дополнительные действия. Пользователь может выбрать заметку в списке и перейти к соответствующей точке на карте либо удалить её. При удалении запись убирается из массива данных и соответствующий маркер удаляется с карты. Состояние заметок сохраняется вместе с данными маршрута, поэтому при повторном открытии планировщика ранее созданные точки могут быть восстановлены. Обработка карты и списка заметок выполняется на стороне клиента, что позволяет изменять содержимое без полной перезагрузки страницы.



5.5. Истории и лайки

Страница историй реализована как динамическая лента публикаций. Данные историй представлены в виде объектов, содержащих идентификатор, автора, направление, текст публикации, изображение, набор тегов и количество

лайков и комментариев. При загрузке страницы JavaScript получает массив публикаций из API и формирует карточки историй. Для каждой записи создаётся единообразная структура карточки, поэтому количество публикаций не зависит от количества заранее написанных HTML-блоков. Для интерактивности карточек реализована система лайков. Кнопка лайка имеет обработчик события click. При первом нажатии состояние публикации меняется на «понравилась», визуальное состояние кнопки изменяется, а значение счётчика увеличивается на единицу. При повторном нажатии действие отменяется, кнопка возвращается в исходное состояние, а счётчик уменьшается. Состояние лайка хранится отдельно для конкретного пользователя, поэтому повторное открытие страницы позволяет восстановить уже выполненные действия. Для этого используются данные авторизованного пользователя и localStorage. При работе с API изменение количества лайков может передаваться на JSON Server, благодаря чему данные публикации сохраняются между загрузками страницы. Таким образом, лента реализует не только статический вывод карточек, но и полный цикл клиентского взаимодействия.



6. Подключение mock API и авторизация

Санкт-Петербург

2026 г.

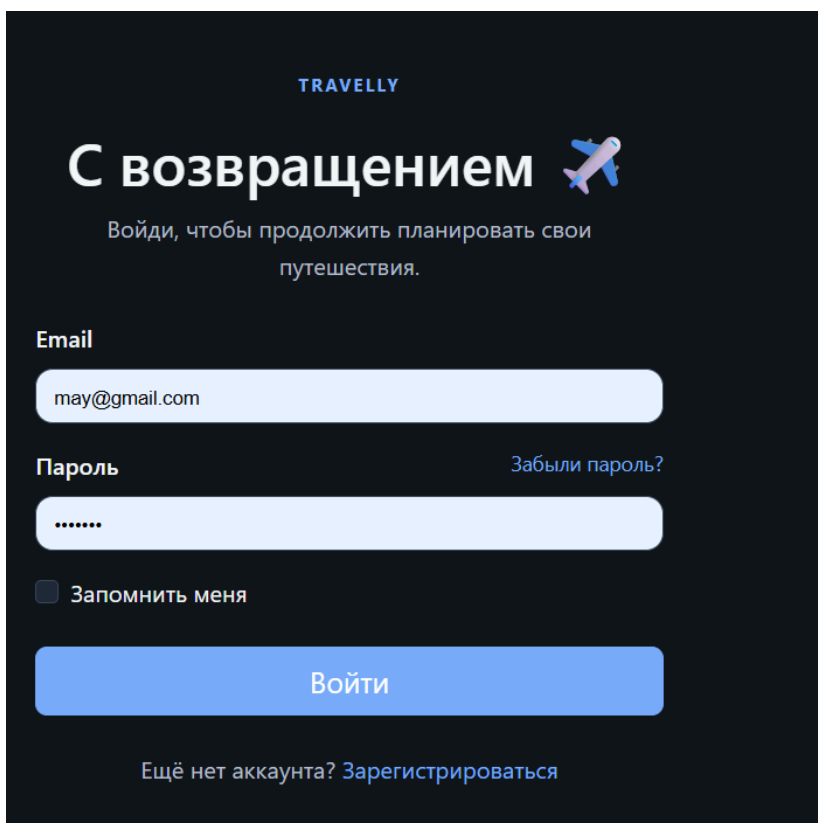
Для имитации backend-части проекта используется JSON Server. Все основные HTTP-операции вынесены в js/api.js. Общая функция request() формирует запрос к адресу http://localhost:3000 и возвращает JSON-ответ.

```
const TravellyAPI = {
  BASE_URL: "http://localhost:3000",

  async request(endpoint, options = {}) {
    const response = await fetch(`${this.BASE_URL}${endpoint}`, {
      headers: { "Content-Type": "application/json" },
      ...options
    });

    if (!response.ok) throw new Error(`API error: ${response.status}`);
    return response.status === 204 ? null : response.json();
  }
};
```

В db.json присутствуют коллекции users, destinations, trips, posts, saved, comments, likes и plans. Это позволяет моделировать пользователей, поездки, истории, сохранённые направления и планы маршрутов. Авторизация реализована в js/auth.js. При входе сначала проверяются локально сохранённые пользователи, затем при доступности JSON Server выполняется поиск пользователя через API. После успешного входа создаётся пользовательская сессия в localStorage. При регистрации пользователь отправляется на сервер через POST-запрос, а при недоступности API используется локальное резервное сохранение.



The screenshot shows the login interface for 'TRAVELLY'. At the top, the brand name 'TRAVELLY' is in blue. Below it, the heading 'С возвращением' (Welcome back) is in white, accompanied by a blue airplane icon. A subtitle in white reads: 'Войди, чтобы продолжить планировать свои путешествия.' (Log in to continue planning your trips). The form includes an 'Email' field with the placeholder 'may@gmail.com', a 'Пароль' (Password) field with masked characters '.....', and a 'Запомнить меня' (Remember me) checkbox. A blue 'Войти' (Log in) button is positioned below the password field. To the right of the password field is a link 'Забыли пароль?' (Forgot password?). At the bottom, there is a link 'Ещё нет аккаунта? Зарегистрироваться' (Don't have an account? Register).

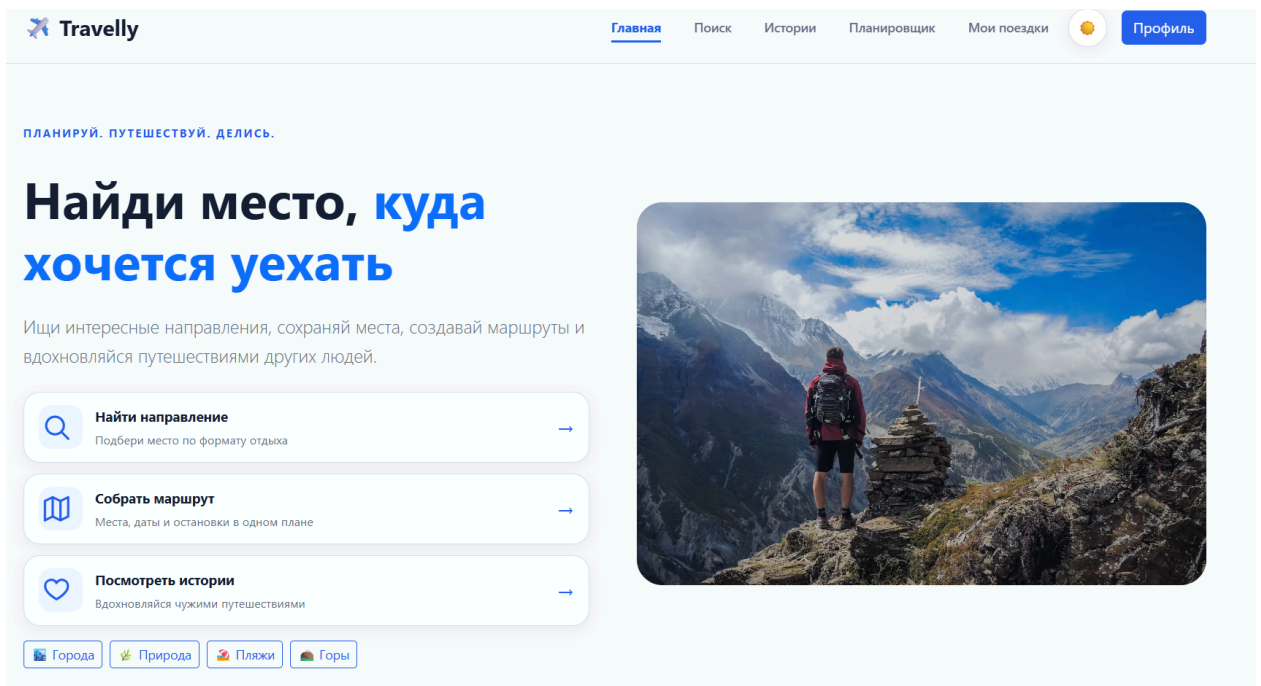
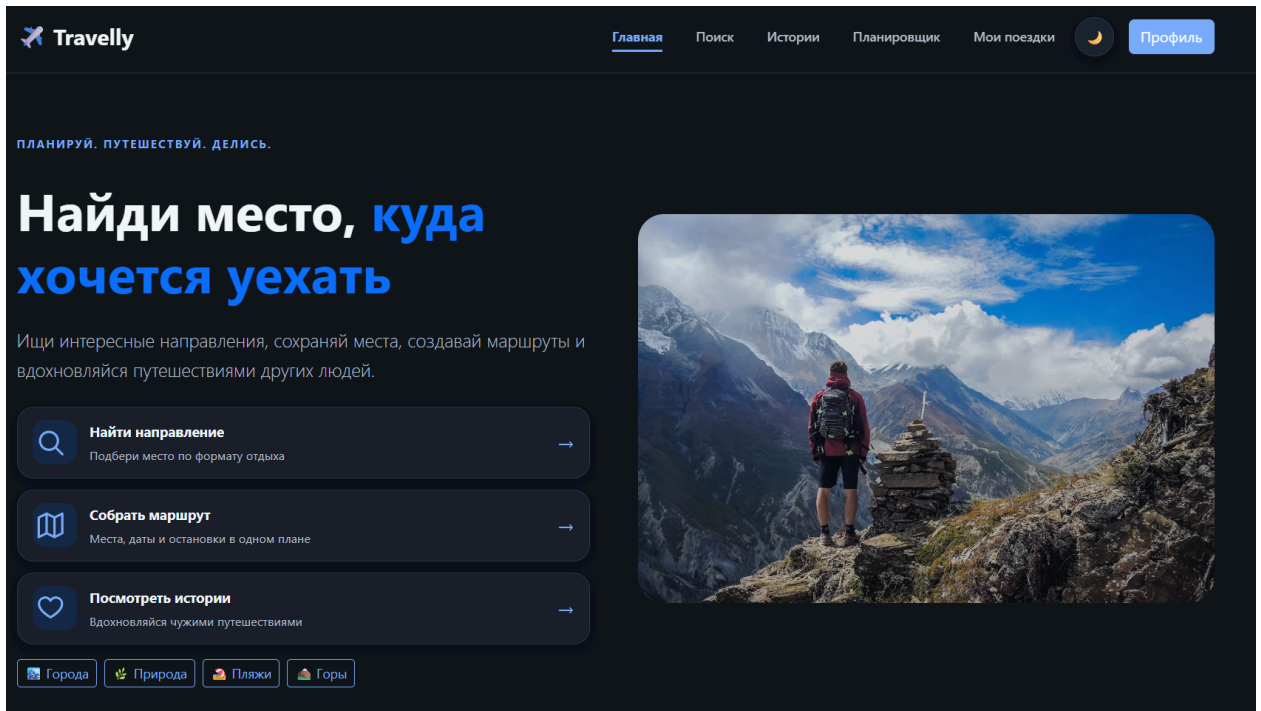
7. Улучшение доступности интерфейса

Для повышения доступности в проект добавлены связанные label для элементов формы, понятные подписи кнопок, alt-тексты изображений, aria-label для интерактивных элементов и aria-live/role="alert" для динамических сообщений. Для фокусируемых элементов предусмотрено видимое состояние focus-visible. Также учитывается настройка prefers-reduced-motion: при включенном параметре пользователя анимации и плавные переходы сокращаются. Это позволяет сделать интерфейс более комфортным для пользователей с чувствительностью к движению.

8. Темизация сайта

В Travelly реализованы светлая и тёмная темы. Цвета вынесены в CSS-переменные, а текущая тема задается атрибутом data-theme у элемента html. JavaScript-модуль theme.js переключает темы, сохраняет выбор в localStorage и при отсутствии ручного выбора учитывает системную настройку prefers-color-scheme.

```
const newTheme = current === "dark" ? "light" : "dark";
applyTheme(newTheme);
localStorage.setItem("travelly-theme", newTheme);
```



9. SVG-спрайт

Для выполнения требования по общему SVG-спрайту создан файл `assets/sprite.svg`. В него вынесены пять иконок: самолёт, поиск, сердце, карта и солнце. Иконки представлены как `symbol` и могут использоваться через SVG `<use>`, поэтому не требуется дублировать исходную разметку на каждой странице.

11. Результат работы

В результате получено цельное веб-приложение Travelly для планирования путешествий. Приложение включает обязательные страницы входа, регистрации, личного кабинета и поиска с фильтрацией, а также дополнительные страницы направлений, историй, планировщика и сохранённых поездок. Клиентская часть взаимодействует с mock REST API через Fetch API и JSON Server. Добавлены интерактивная карта, пользовательские заметки, лайки, светлая и тёмная темы и общий SVG-спрайт. В ходе лабораторной работы был разработан и доработан сайт Travelly по выбранному варианту приложения для планирования путешествий и маршрутов. Реализованы обязательные элементы, подключено взаимодействие с API, добавлены пользовательские сценарии, темизация и меры по улучшению доступности.